

Portfolio

이 다 산

서비스를 직접 만들고 운영하며
실제 사용 과정에서 발견한 문제를 해결해봤습니다.

세 프로젝트에서 쌓은 경험

01 Gimpo-Wellness

태그·거리 기반 코스 추천을 구현하고, 코스 생성과 AI 설명을 분리해 응답 대기 시간을 줄였습니다.

02 ReviewMate

GitHub PR 자동 리뷰 과정에서 비동기와 트랜잭션을 설계했습니다.

03 Plogues

플로깅 모임의 동시성 제어와 센서 데이터 조회 성능을 개선했습니다.

김포의 장소를 나만의 여행 코스로

2026.08-09 · 팀 4명 · 프론트·백엔드 개발

김포시 관광진흥팀 인터뷰에서 도출한 요구사항을 바탕으로 만든 지도 기반 서비스



담당 역할

순례길 콘셉트 여행 코스 기능 개발
고정 코스·이미지 라이선스 관리
AWS · Docker 배포
Promethues · Grafana Cloud 모니터링

기술

Spring Boot · MyBatis · Oracle
React · Prometheus · Grafana

김포시 관광진흥팀 인터뷰에서 나온 요구

기여: 인터뷰 요구사항을 바탕으로 각 대응 방안을 제안

인터뷰에서 확인한 요구

대응 방안

애기봉·대명항 주변 음식점을
방문객에게 더 알리고 싶다

음식점 자료를 확보·검토해
지도에서 주변 정보를 제공

소개된 식당이 실제로 있는지
확인하기 어렵다

식품안전과에 영업신고 식당 자료를
요청하고, 이를 바탕으로 데이터 검토

조각공원과 전시 작품의
홍보가 부족하다

관광지 TOP 10 노출과
작품 소개 연결 방식 검토

단체 관광객이 함께 식사할
음식점을 찾기 어렵다

수용 인원 데이터 미확보로
단체 수용 태그는 보류

태그와 거리로 경유 관광지 10개 추천

선택 태그가 포함된 장소 중 경로와의 최소거리 2km 이내 후보 추천

The screenshot displays the '순례자의 길 제작' (Pilgrimage Route Creation) interface. On the left, there are filters for '태그' (Tags) including '해양' (Coastal), '반려동물' (Pet-friendly), '레저' (Leisure), and '쇼핑' (Shopping). Below these is a '추천받기' (Receive Recommendation) button and a '중간 관광지 추가하기' (Add Intermediate Attraction) section. The '중간 관광지 추가하기' section lists three potential stops: 1. 김포중앙공원 (Gimpo Central Park), 2. 김포레코파크 (Gimpo ReCo Park), and 3. 김포 장릉 (Gimpo Jangryung). On the right, a map shows a recommended route (green line) starting from a blue dot and ending at a red dot, passing through several points of interest marked with orange dots. The map includes labels for various locations like '김포성당' (Gimpo Cathedral), '김포제일교회' (Gimpo Jeil Church), '김포5일장' (Gimpo 5-day Market), and '김포초등학교' (Gimpo Elementary School).

01 태그 유사도

일치 태그 수 ÷ 선택 태그 수
5개 중 4개가 일치하면 0.8

02 경로 거리 점수

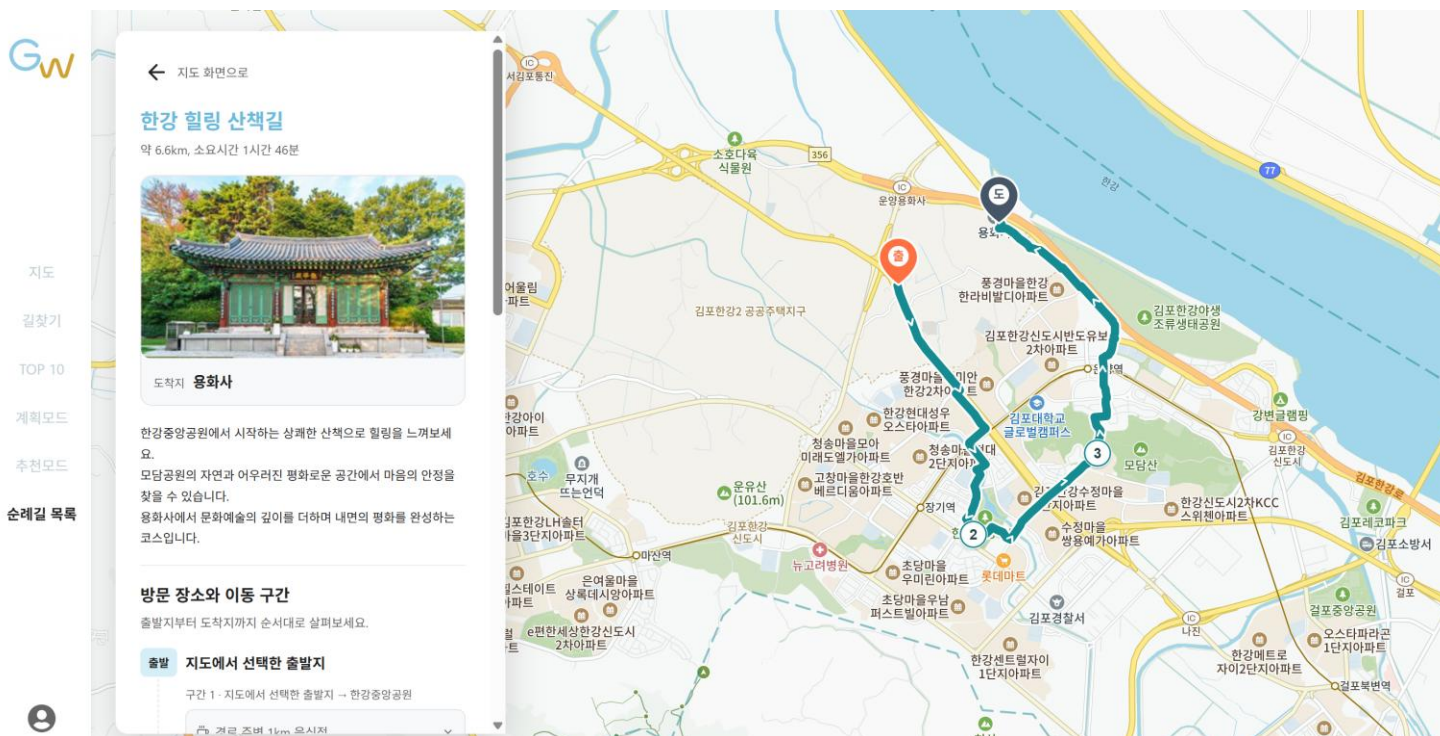
≤500m 1.0 · ≤1km 0.7
≤1.5km 0.5 · ≤2km 0.2

03 최종 점수와 선별

태그×0.7 + 거리×0.3, 상위 10개

코스과 설명을 분리해 대기 시간 개선

코스를 먼저 반환하고, AI 설명은 별도로 생성



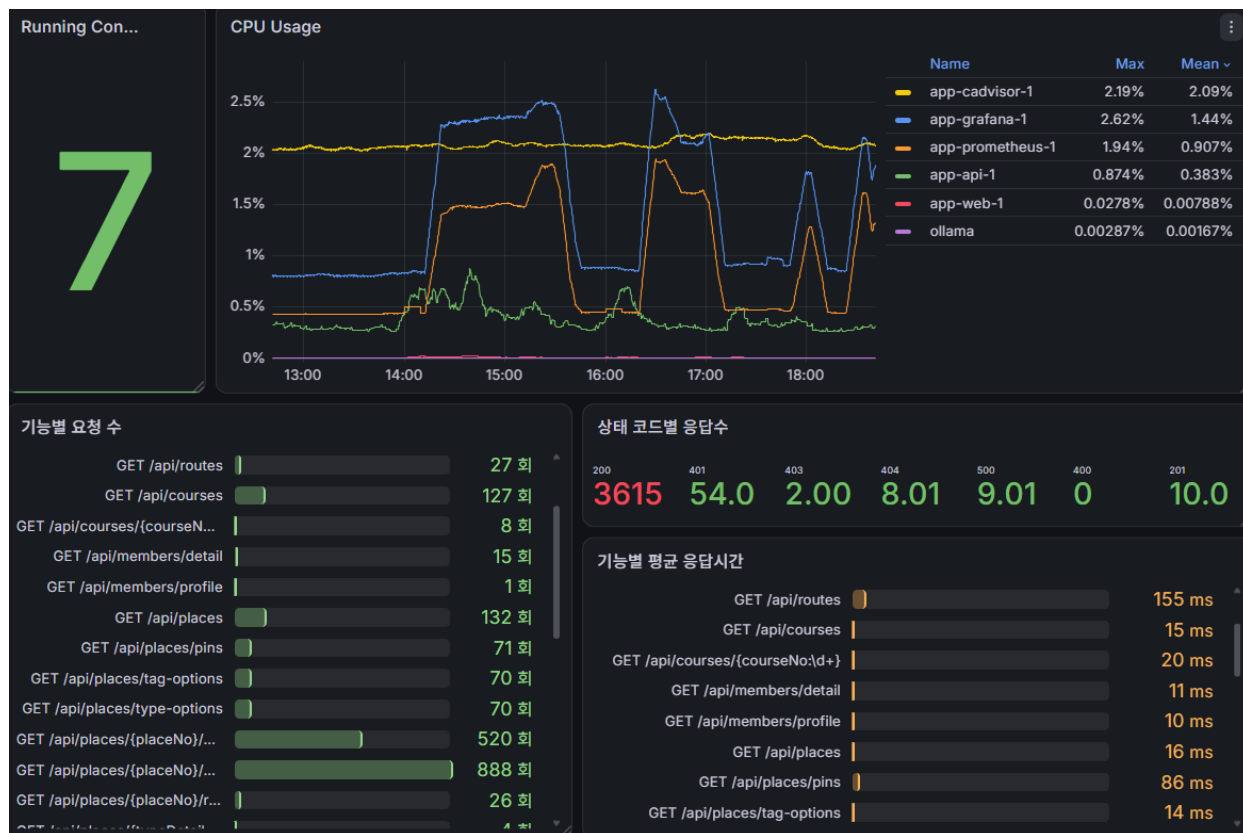
01 코스·설명 API 분리

문제 : 설명이 끝날 때까지 기다리던 응답
 해결 : API를 분리해 코스를 먼저 확인
 결과 : 출발/도착지 선택 시 코스부터 확인

02 설명 생성 모델 전환

문제 : AWS 환경, Ollama의 응답이 느림
 해결 : GPT-4.1 mini API로 전환
 결과 : 응답시간 17.4초 -> 1.9초

서비스 상태와 HTTP 요청 지표 수집



베타테스트에서 발견한 500 오류를 코드의 원인까지 추적

관측 Grafana에서 상태 코드별 응답 확인 및 실패 요청 식별

로그 추적 코스 수정 요청의 ORA-00001 로그에서 중복 키 충돌 확인

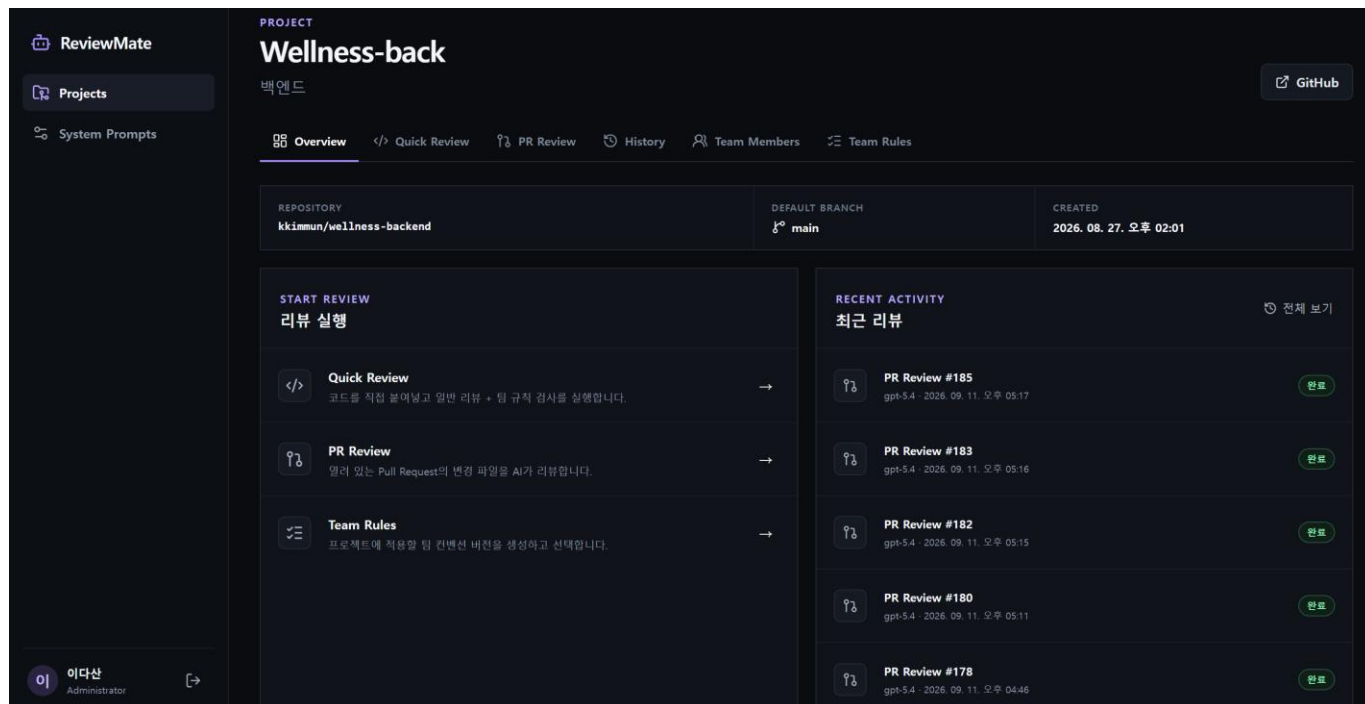
코드 수정 주식 처리된 기존 목록 삭제(DELETE) 호출 복원

확인 테스트를 통한 수정 결과 검증

PR 리뷰에 들던 시간을 줄이고 싶었습니다

2026.07-08 · 팀 1명 · 백엔드 개발

팀 개발의 리뷰 부담에서 시작한 AI 코드 리뷰 서비스



기능

직접 코드 입력 코드 리뷰
Github Webhook, 자동 PR 리뷰
관리자 시스템 프롬프트 관리
팀장 팀원 · 팀 컨벤션 관리

기술

Java 21 · Spring Boot · JPA
PostgreSQL
GitHub API · OpenAI API

리뷰를 읽고, 수정으로 이어진 순간

웰니스 PR에 적용한 자동 리뷰 · wellness-frontend #83

src/features/map/components/RoutePanel.styles.js
Hide resolved

```

125 136 border-radius: ${theme.radius.lg};
126 137 background: ${theme.colors.bgLight};
138 +
139 + @media (max-width: 768px) {

```

ham-zi 5 days ago
Collaborator

⚠ 위험 · 스타일 파일에 머지 충돌 마커가 남아있음

머지 충돌 마커가 소스에 남아 있어 번들링 또는 파싱 단계에서 즉시 실패해 해당 변경이 포함된 빌드를 실행할 수 없습니다.

제안: 충돌 마커를 제거하고 `PointRow` 의 최종 레이아웃 구현 하나만 남기세요.

peony639-lab 5 days ago
Collaborator Author

수정했습니다

peony639-lab 5 days ago
Collaborator Author

주의 부분은 회원탈퇴 처리 헤더에서 프로필 누르면 바로 보이던 부분 없앤거라 그대로 진행해도 됩니다

코드 옆에서 확인

긴 리뷰 본문 대신
문제 코드 옆에 직접 표시

실제 사용 흐름

충돌 마커 지적에
작성자가 수정 완료 응답

불편을 발견할 때마다 처리 방식을 바꿨습니다

문제	바꾼 방식	달라진 점
웹에서 브랜치를 고르고 diff 리뷰를 직접 요청	Webhook으로 PR 이벤트와 리뷰 연결	PR 생성과 추가 push 시 자동으로 리뷰 실행
댓글에 리뷰가 남겨져, 리뷰를 읽고 문제 코드를 찾으러 가야함	변경된 코드 줄에 인라인 리뷰 등록	문제 코드와 수정 의견을 같은 위치에서 확인
LLM 응답이 끝날 때까지 HTTP 요청이 유지됨	접수와 AI 처리를 분리 커밋 후 비동기 실행	reviewId를 먼저 반환 완료 상태는 별도로 관리
일반 리뷰와 팀 규칙을 한 번에 맡기며 역할 혼동	일반·규칙 리뷰의 프롬프트와 호출 분리	결과와 프롬프트 버전을 역할별로 추적·관리

DTO로 비동기 작업의 LAZY 로딩 의존성 제거

비동기 작업에서 LAZY 연관관계를 읽을 수 없었습니다

문제 비동기 리뷰 중 엔티티의 LAZY 연관관계에 접근하면 예외가 발생했습니다.

원인 영속성 컨텍스트가 종료된 엔티티를 비동기 작업에서 그대로 사용했습니다.

해결 트랜잭션 안에서 필요한 값을 DTO로 추출한 뒤, 외부 API 호출에 전달했습니다.

결과 비동기 작업의 LAZY 로딩 의존성을 제거하고 DB 작업과 외부 호출을 분리했습니다.

플로깅 모임과 환경 데이터를 연결

2026.06-07 · 팀 5명 · 팀장·프론트·백엔드 개발

탄소중립 활동으로 플로깅/식목 모임을 생성하고 참여하는 커뮤니티 사이트



담당

참가 신청·승인 흐름
마이페이지 · 관리 기능
식목지 환경 변화 센서 API(라즈베리파이)

기술

Spring Boot · MyBatis · Oracle
React · AWS · Docker

동시 승인 시 정원 초과 문제를 해결했습니다

프로젝트 완성 후 동시 요청 테스트에서 발견한 문제

문제 여러 승인 요청이 같은 잔여 정원을 읽어 정원을 초과할 수 있었습니다.

비교 검토한 낙관적 락 적용안은 현재 인원 컬럼과 로직 추가가 필요했습니다.
비관적 락은 잠금 대기가 있지만 기존 인원 조회 로직을 유지할 수 있었습니다.

선택 정원 준수와 구현의 단순함을 우선해 비관적 락을 선택했습니다.
모집글 행을 잠근 뒤 같은 트랜잭션에서 정원과 신청 상태를 다시 확인했습니다.

검증 H2의 Oracle 호환 모드에서 동시 승인·중복 승인 테스트 코드를 작성했습니다.

반복 조회의 비용을 캐시로 줄였습니다

라즈베리파이 센서 집계 결과를 재사용하고, 10분 간격으로 갱신

적용 전

**조회 요청마다 DB에서
AVG·GROUP BY 집계**

같은 조회가 몰릴 때마다
최근 8시간 데이터를 다시 집계

적용 후

**집계 결과를 Caffeine에 저장
같은 키의 요청에 재사용**

10분 간격으로 비우고 다음 조회에 갱신
별도 캐시 서버가 필요하지 않아 Caffeine 선택

평균 응답 시간 약 83% 감소

로컬 k6 테스트에서 캐시 적용 전후를 비교

측정 항목	캐시 OFF	캐시 ON
평균 응답 시간	35.83 ms	6.04 ms
p95 응답 시간	58.65 ms	13.75 ms
처리량	1,386 req/s	7,268 req/s

VU 50 · 1분 · 로컬 API · 두 조건 모두 요청 실패율 0%

단일 테스트 환경의 측정값이며 실제 운영 성능을 보장하는 수치는 아닙니다.

읽어주셔서 감사합니다.

이 다 산

신입 개발자

Email ddmt0805@gmail.com

GitHub github.com/ham-zi

Resume ham-zi.github.io